



Faculty of Computers and Artificial Intelligence

Cairo University

IS313 – Data Warehouse

Data Warehouse Project

Section: IS S1,2

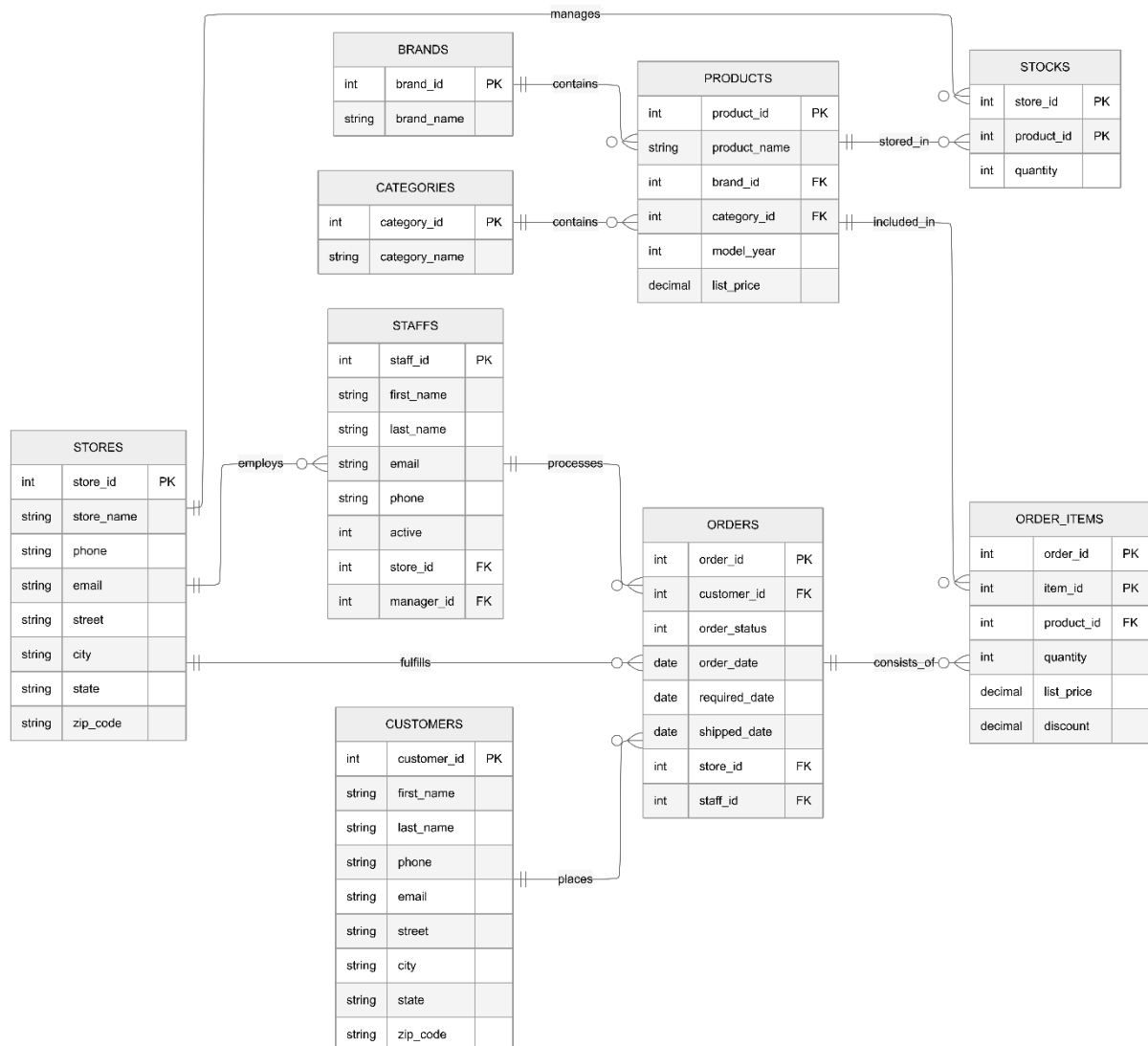
Name	ID
Nourhan Mohammed Ahmed	20230560
Habiba Mahmoud Mohammed	20230119
Basmala Mamdouh Esmail	20230092
Mohannad Salah	20231182
Adham Ghallab	20230045

Data set source: Kaggle

Dataset: Bike Store Relational Database | SQL

Deliverables:

1. Physical model of the source system (the tables from which you will build your model).



- **Source Description:**

- **STG_Brands:** Stores the manufacturer names for the bicycles.
Columns: `brand_id`, `brand_name`
- **STG_Categories:** Defines the different types of bicycles available.
Columns: `category_id`, `category_name`

- **STG_Customers:** Contains profile and contact information for all registered customers.
Columns: customer_id, first_name, last_name, phone, email, street, city, state, zip_code
- **STG_Order_Items:** Lists specific product details, pricing, and discounts for every order.
Columns: order_id, item_id, product_id, quantity, list_price, discount
- **STG_Orders:** Tracks the header-level details including order status and key fulfillment dates.
Columns: order_id, customer_id, order_status, order_date, required_date, shipped_date, store_id, staff_id
- **STG_Products:** Serves as the master catalog for all bicycle models and their specifications.
Columns: product_id, product_name, brand_id, category_id, model_year, list_price
- **STG_Staffs:** Maintains records of employees, their roles, and their assigned store locations.
Columns: staff_id, first_name, last_name, email, phone, active, store_id, manager_id
- **STG_Stocks:** Records the current inventory levels for every product across different store locations.
Columns: store_id, product_id, quantity
- **STG_Stores:** Contains information and locations for the physical retail outlets.
Columns: store_id, store_name, phone, email, street, city, state, zip_code

2. Dimensional model

- Business Processes Modeled

We have selected three core business processes from the Bike Store system to provide a 360-degree view of operations:

- **Sales Transactions:** Tracking the individual movement of bicycles from the store to the customer. Using list_price and discount from order_items.csv.
- **Shipping & Logistics:** Monitoring delivery performance by comparing required_date and shipped_date from the orders.csv file, and calculates the late delivery risks
- **Order Management & Financials:** Analyzing order success and store-level performance using order_status and store_id. It summarizes the overall financial benefit of transactions across different store locations and staff members.

- **Grain of the Fact Tables**

The grain defines the level of detail for a single row in each fact table:

- **Fact_Sales:** One row per item within a specific order (Atomic/Line-item grain). This allows for the highest level of analytical granularity.
- **Fact_Shipping:** One row per shipment transaction for a given order and store location.
- **Fact_Orders:** One row per unique Order ID (Header level grain). This summarizes the entire transaction regardless of how many items were purchased.

- **Type of Each Fact Table**

1) **Fact_Sales**

- **Type:** Transactional Fact Table.
- Records specific sales events to track individual product and revenue generation at the line-item level.
- **KPIs:**
 - **Total Net Revenue per Brand:** The sum of SalesAmount minus Discount to show actual earnings per brand.
 - **Discount Rate Percentage:** The total discount value divided by total SalesAmount to monitor price reductions.
 - **Top Selling Products per quantity:** A ranking based on the total Quantity sold to identify popular items.
 - **Revenue per Brand:** The total earnings generated by specific manufacturers to identify high-value brands.

2) **Fact_Shipping**

- **Type:** Transactional Fact Table.
- Records the shipping lifecycle to measure delivery performance.
- **KPIs:**
 - **Late Shipment Rate per Store:** The percentage of total orders per store where delivery occurred after the required date.
 - **Average Fulfillment Time:** The average number of days taken to process and ship an order from the date it was placed.

3) Fact_Orders

- **Type:** Transactional Fact Table.
- Captures the summarized financial and status outcome of a transaction to provide a view of overall business health.
- **KPIs:**
 - **Average Order Value:** The total order profit divided by the total number of unique orders.
 - **Order Success Rate:** The percentage of transactions that reached a completed status without being cancelled or rejected. As well as the percentage of orders that were rejected, processed and rejected to help in comparison.
 - **Average items count:** The average number of items contained within a single transaction.

- Dimensions and Their Types

We implemented 6 Dimension tables using an SCD Type 1 , SCD Type 0(Slowly Changing Dimension) and Static strategy.

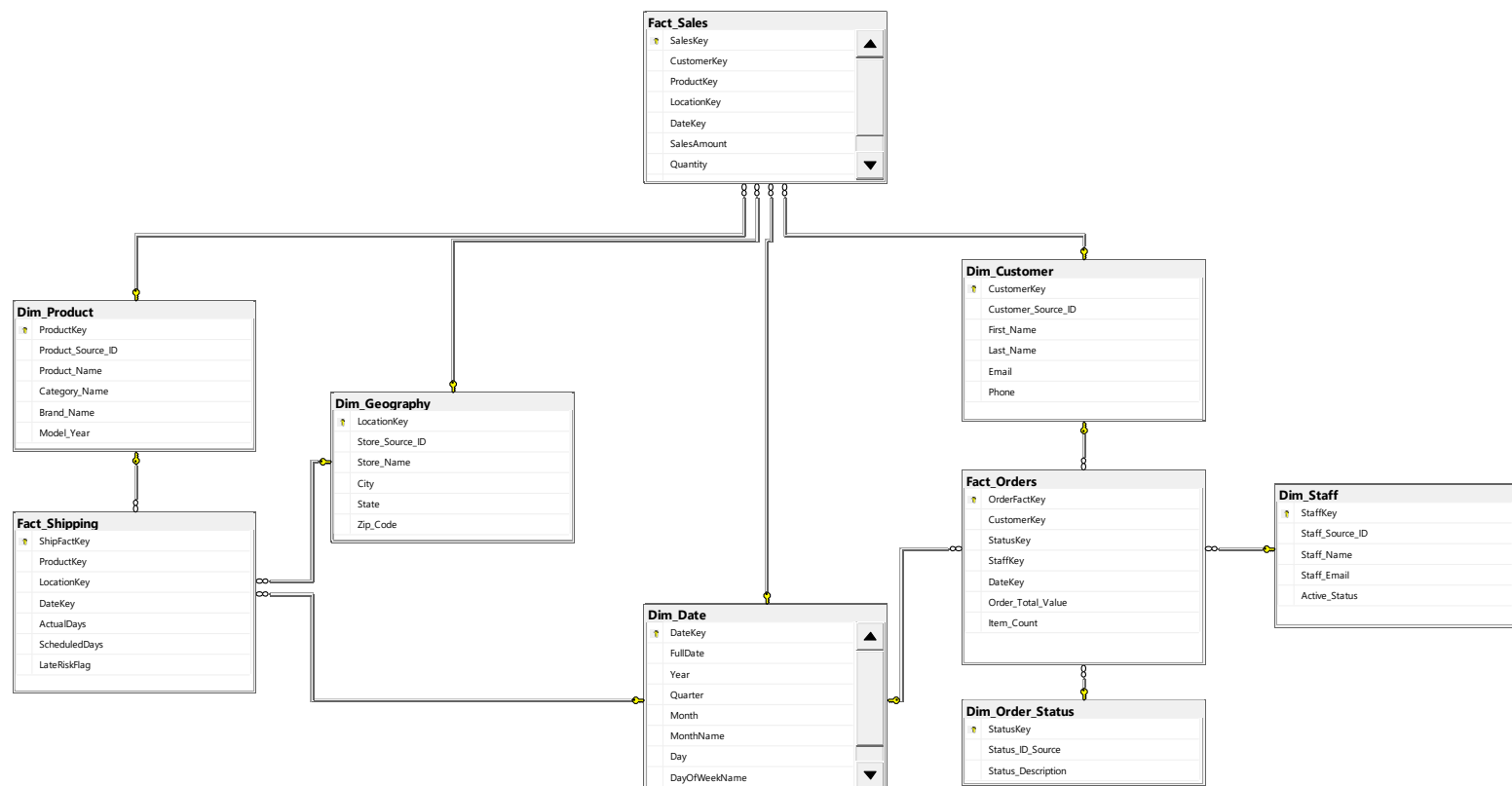
Dimension Name	Type	Description
Dim_Date	Static / Generated	A generated time dimension based on the order_date, required_date and shipped_date covering years 2016–2018.
Dim_Customer	SCD Type 1	Contains customer names and contact details. It overwrites the old data as the customer data doesn't change frequently so there is no need to keep history.
Dim_Product	SCD Type 2	Stores product names, categories, brands, and model years. Maintains full history by expiring the old record and inserting a new version whenever a change is detected.
Dim_Geography	SCD Type 1	Maps store and the customer locations for the orders.
Dim_Staff	SCD Type 2	Stores staff names, emails, and active status. Maintains full history by expiring the old record and inserting a new version whenever staff details change.
Dim_Order_Status	Static	Acts as a fixed lookup table for the order lifecycle, mapping status codes to descriptions: Pending, Processing, Rejected, and Completed.

- Measures and Data Types

Measures are the quantitative values stored in fact tables used for KPI calculations:

Fact Table	Measure	Data Type	Description
Fact_Sales	SalesAmount	Decimal(18,2)	Total revenue for the item. Quantity * list_price
Fact_Sales	Quantity	Integer	Number of units sold.
Fact_Sales	Discount	Decimal(18,2)	Discount value applied.
Fact_Shipping	ActualDays	Integer	Real time taken for delivery. Difference between shipped_date and order_date
Fact_Shipping	ScheduledDays	Integer	Promised delivery time. Orderdate – requireddate
Fact_Shipping	LateRiskFlag	Integer (0/1)	Binary indicator of late delivery. If shippeddate>requireddate
Fact_Orders	Order_Total_Value	Decimal(18,2)	Total profit per order.
Fact_Orders	Item_Count	Integer	It counts the number of unique items within a single order_id

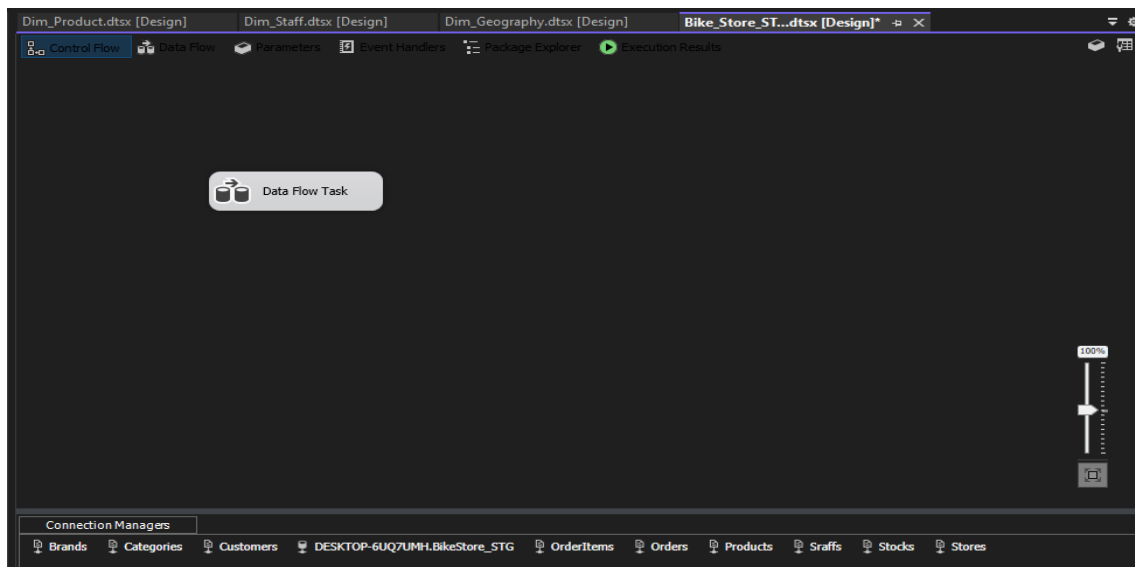
- Galaxy Schema Model:



3. Screenshots of the data flow tasks, and control flow tasks used for building the DWH

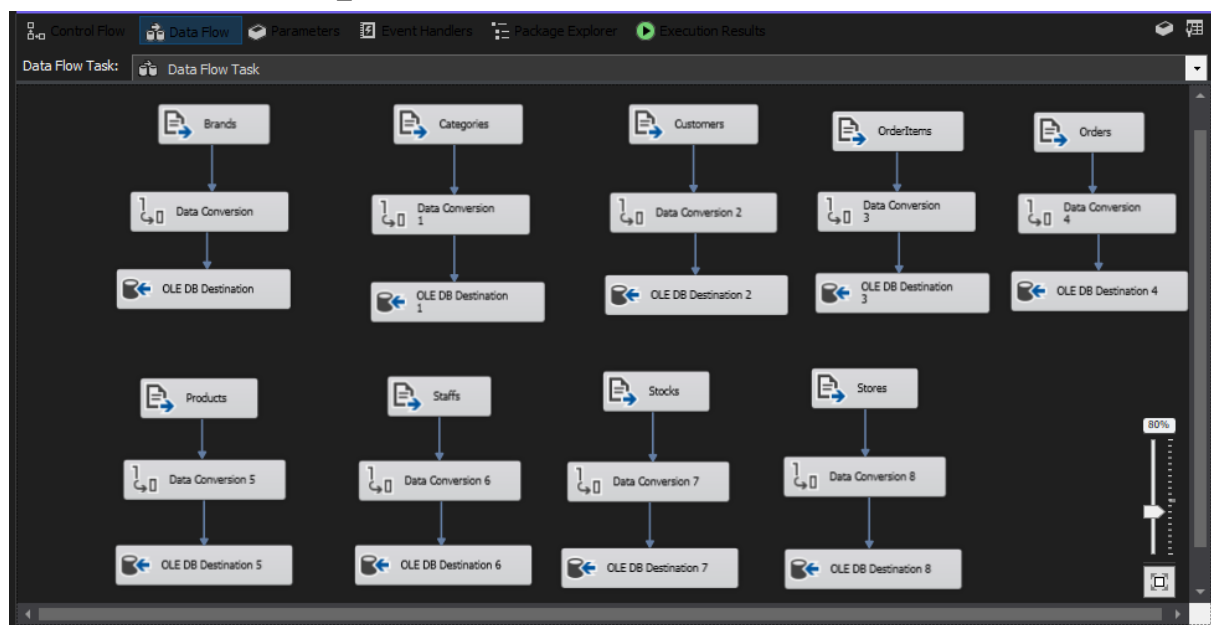
a) Bike_Store_STG:

- **Flow Type:** Control Flow
- **Title:** STG Load — CSV Flat Files to Staging (9 Tables)



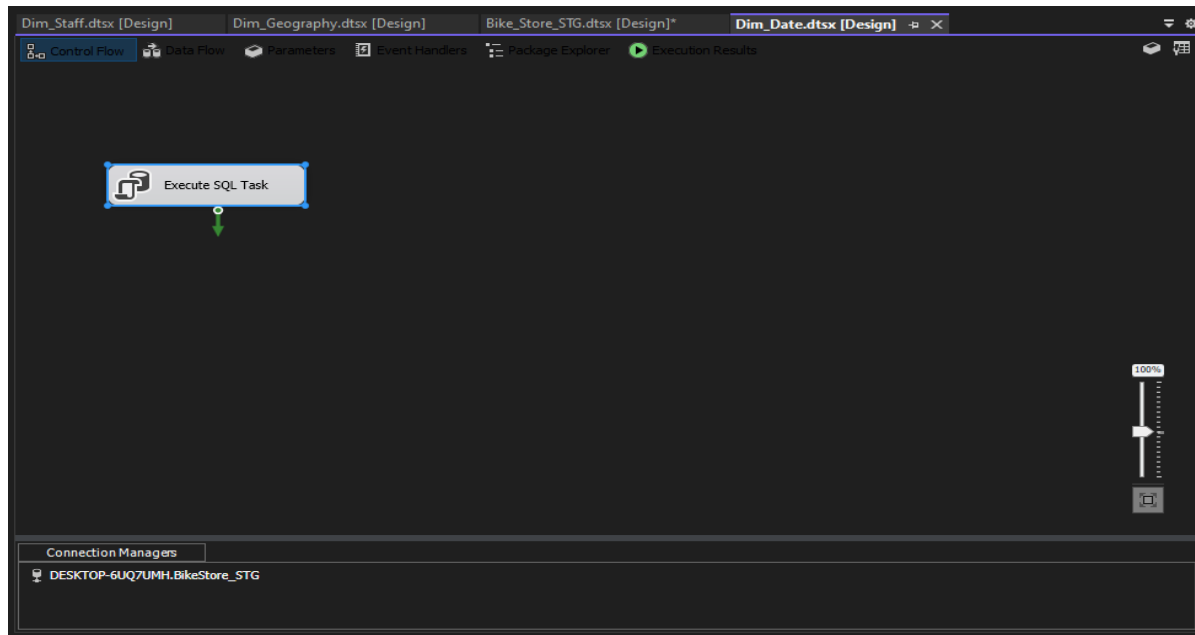
b) Bike_Store_STG:

- **Flow Type:** Data Flow
- **Title:** STG Load — Data Flow Overview
- **Description:** Loading the source, cleaning and placing them in the BikeStore_STG database



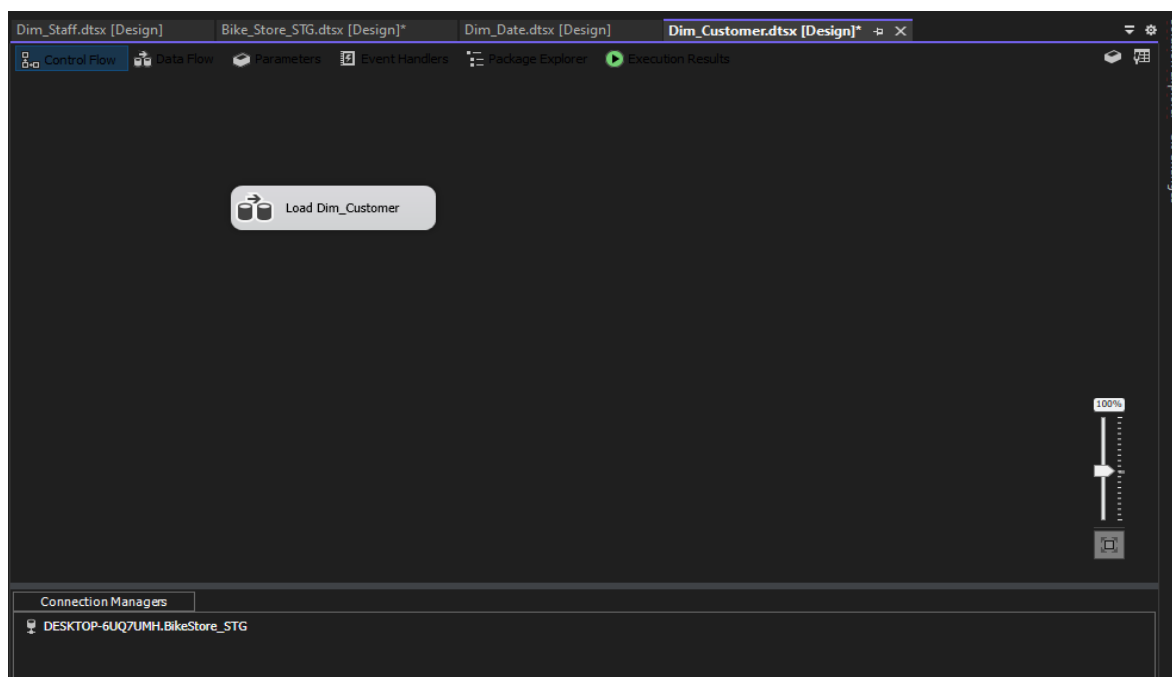
c) Dim_Date.dtsx:

- **Flow Type:** Control Flow
- **Title:** Dim_Date — Execute SQL Task to Populate Date Dimension
- **Description:** It is built once via an SQL WHILE loop to build and populate the date dimension. It iterates day-by-day from 2016-01-01 to 2018-12-31. Populates DateKey (int), FullDate, Year, Quarter, Month, MonthName, Day, DayOfWeekName, IsWeekend. No SSIS Data Flow needed as historical dates never change.



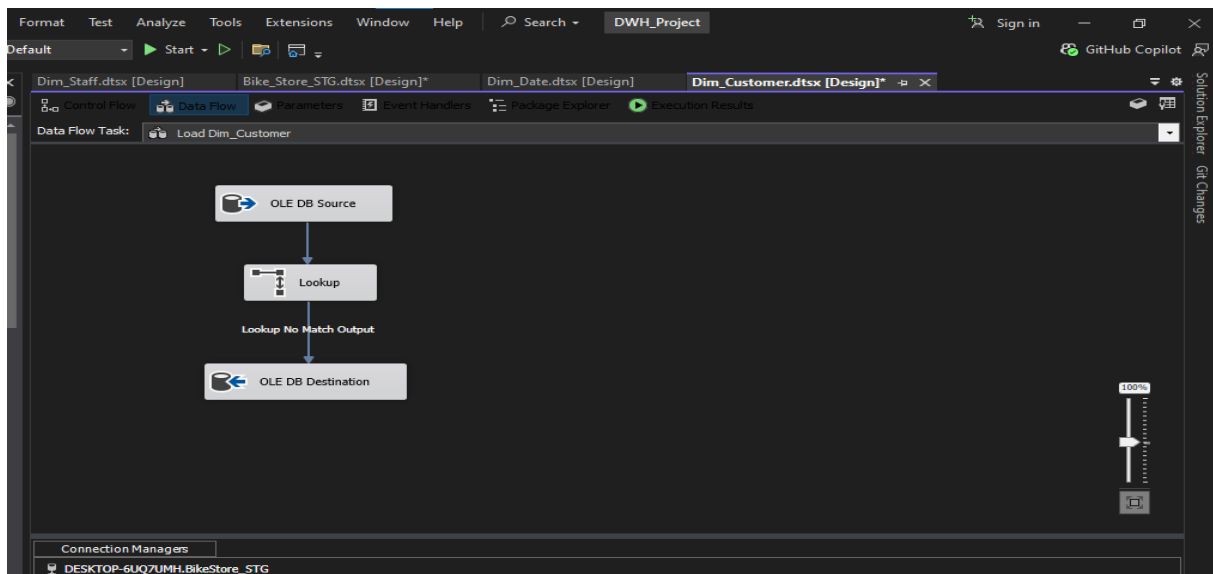
d) Dim_Customer.dtsx

- **Flow Type:** Control Flow
- **Title:** Dim_Customer — Control Flow Overview



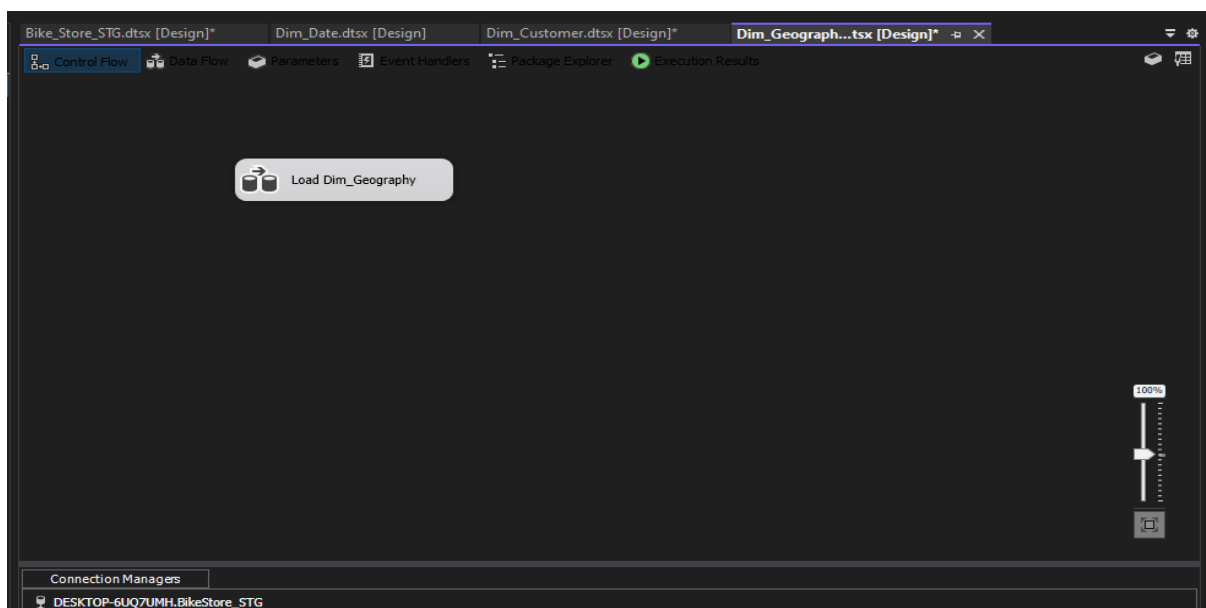
e) **Dim_Customer.dtsx:**

- **Flow Type:** Data Flow
- **Title:** Dim_Customer — SCD Type 1: Insert New Customers Only
- **Description:** The source is STG_customers (customer_id, first_name, last_name, email, phone). Lookup matches on Customer_Source_ID. Only the No Match Output is routed to the OLE DB Destination (new customers inserted). Match Output is discarded as existing records are never updated. This is done using SCD Type 1 insert-only pattern.



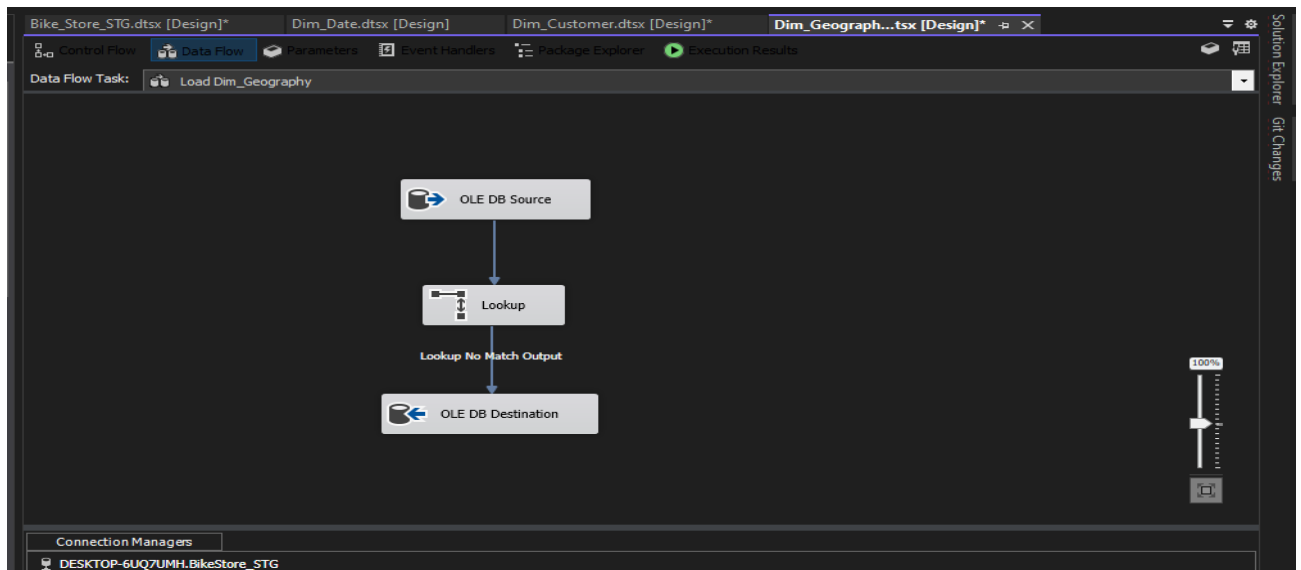
f) **Dim_Geography.dtsx:**

- **Flow Type:** Control Flow
- **Title:** Dim_Geography — Control Flow Overview



g) Dim_Geography.dtsx:

- **Flow Type:** Data Flow
- **Title:** Dim_Geography — SCD Type 1: Insert New Stores Only
- **Description:** The source is STG_stores (store_id, store_name, city, state, zip_code). Lookup matches on Store_Source_ID. Only the No Match Output records are routed to destination. Store location data is static in the BikeStore dataset as there are no store relocations nor will the current data set change, so we only insert and don't update.



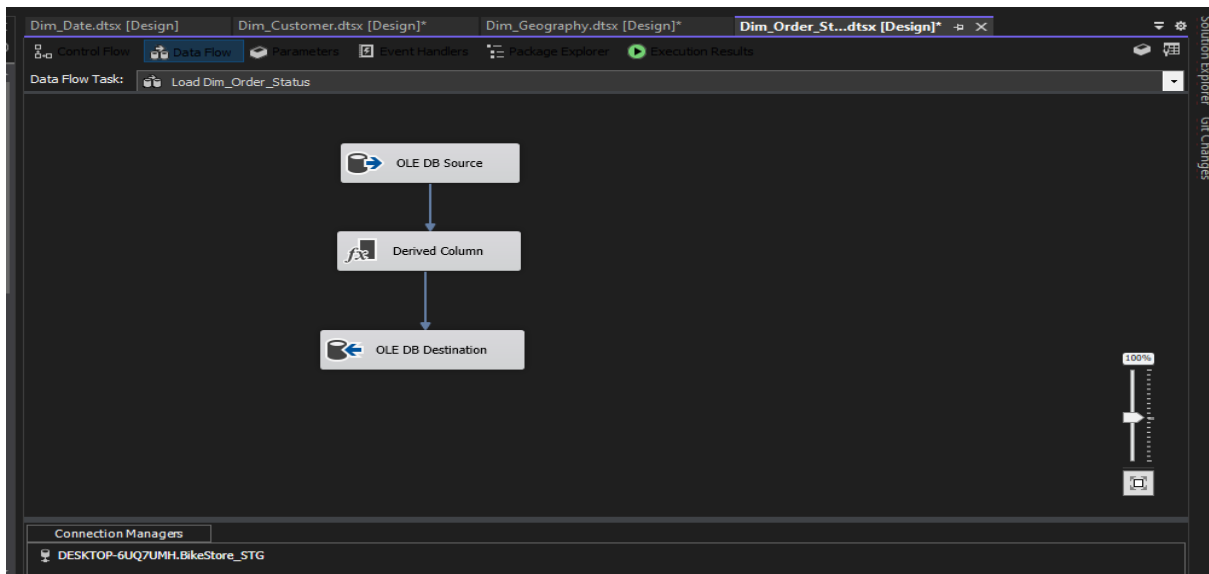
h) Dim_Order_Status.dtsx:

- **Flow Type:** Control Flow
- **Title:** Dim_Order_Status — Control Flow Overview



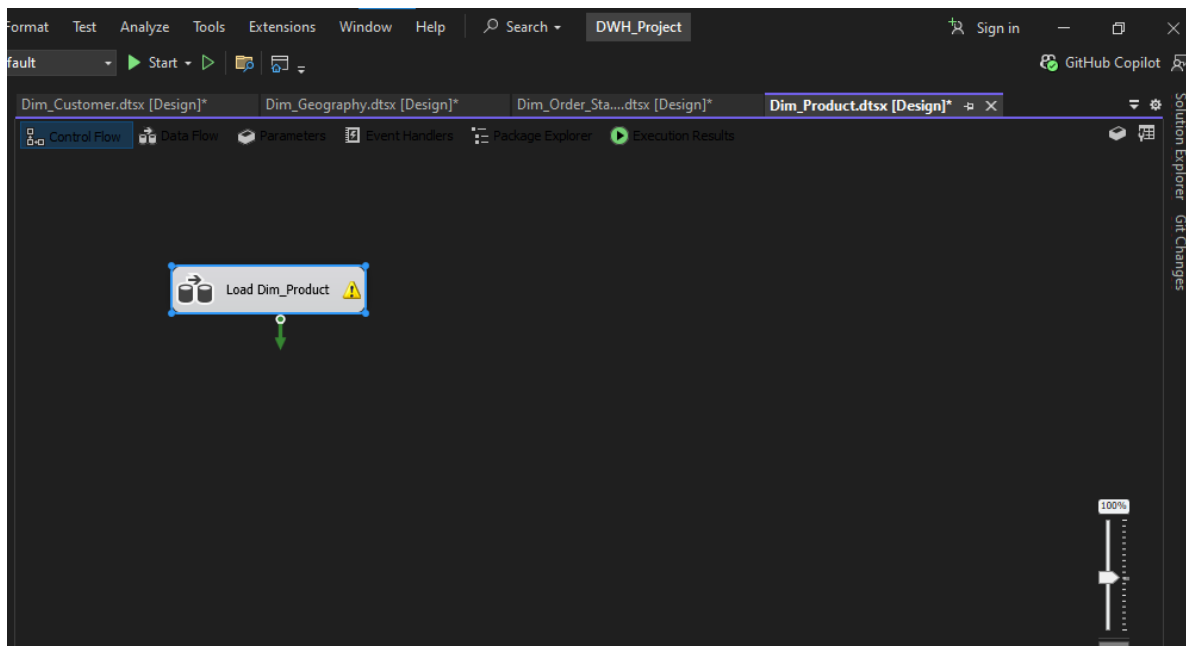
i) **Dim_Order_Status.dtsx:**

- **Flow Type:** Data Flow
- **Title:** Dim_Order_Status — Static Load with Derived Status Descriptions
- **Description:** The source is the 'DISTINCT' order_status values from STG_orders. Derived Column maps the integer codes to text descriptions: 1→"Pending", 2→"Processing", 3→"Rejected", else→"Completed" and this produces Status_Description column. Static lookup table — no versioning needed.



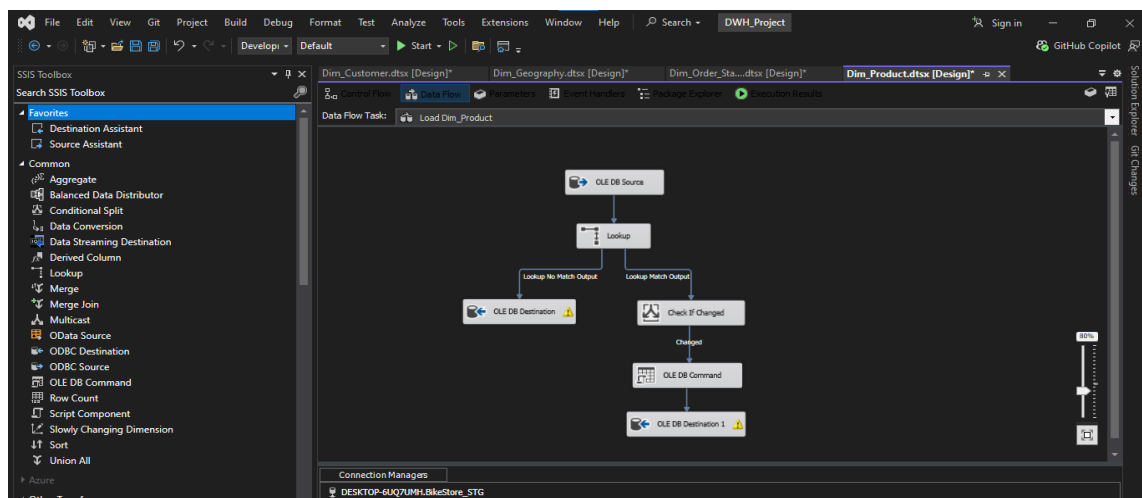
j) **Dim_Product.dtsx:**

- **Flow Type:** Control Flow
- **Title:** Dim_Product — Control Flow Overview



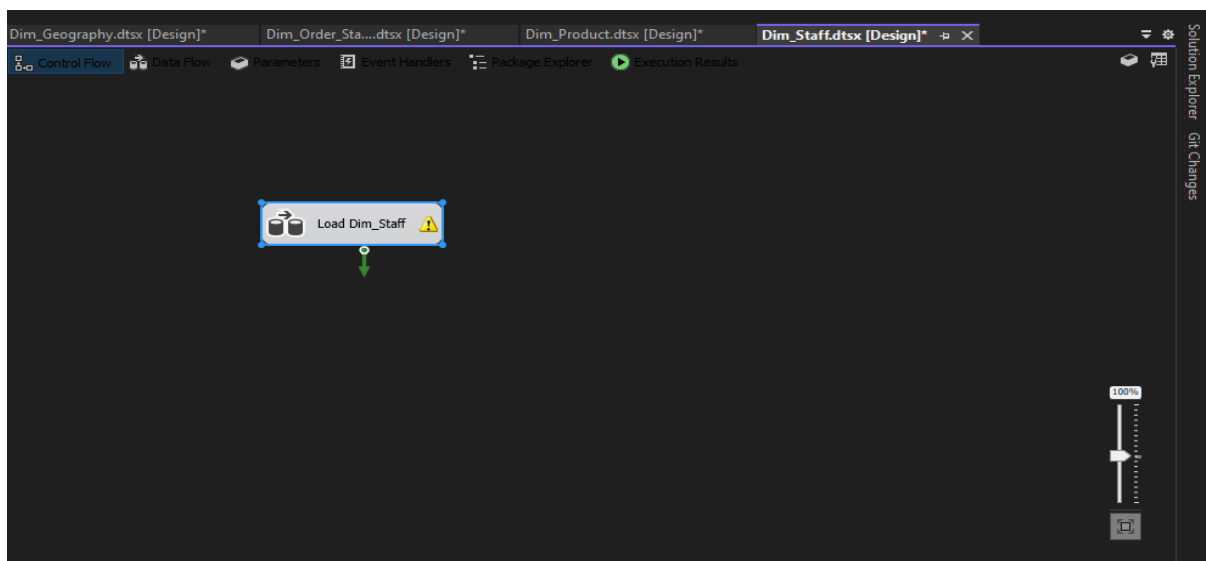
k) Dim_Product.dtsx:

- **Flow Type:** Data Flow
- **Title:** Dim_Product — SCD Type 2: Full Version Tracking
- **Description:** The source is STG_products JOIN STG_categories JOIN STG_brands. Lookup matches on Product_Source_ID. If there is no Match then insert new row (new product). Else if there is a match and something changed (product_name OR category_name OR brand_name OR model_year) then the OLE DB Command runs UPDATE to expire old row (EndDate=GETDATE(), IsCurrent=0), then new row inserted with IsCurrent=1. Else if there is a match and no change then the row is discarded. DDL includes StartDate, EndDate, IsCurrent columns on Dim_Product.



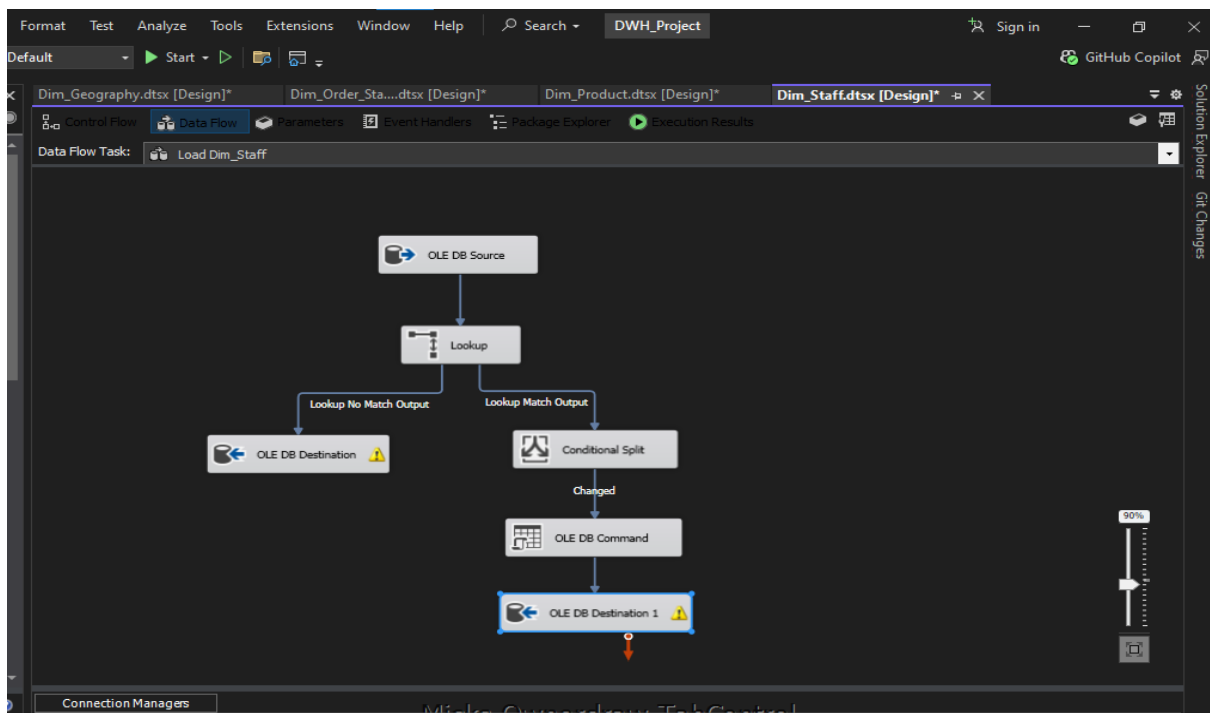
l) Dim_Staff.dtsx:

- **Flow Type:** Control Flow
- **Title:** Dim_Staff — Control Flow Overview

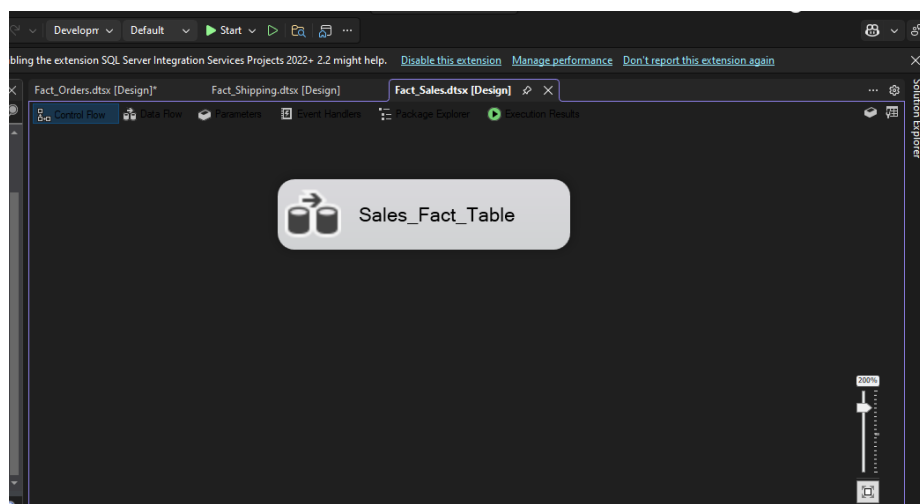


m) Dim_Staff.dtsx:

- **Flow Type:** Data Flow
- **Title:** Dim_Staff — SCD Type 2: Full Version Tracking with ISNULL Handling
- **Description:** The source is STG_staffs with ISNULL handling (first_name+last_name → Staff_Name, email → Staff_Email, active → Active_Status). Lookup matches on Staff_Source_ID. If there is no match then insert new row. Else if there is a match and has been changed (Staff_Name OR Staff_Email OR Active_Status), then expire old row and insert the new one. Else if there was a match and wasn't changed then discard the row. DDL includes StartDate, EndDate, IsCurrent columns on Dim_Staff.

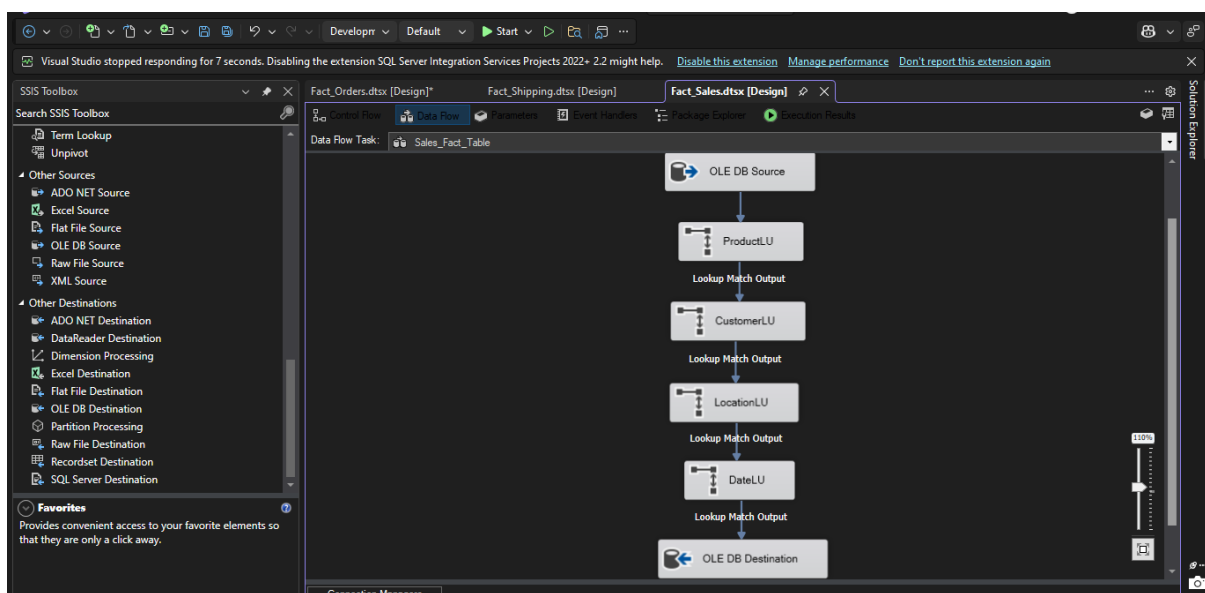


n) Fact_Sales.dtsx → Flow Type is Control flow



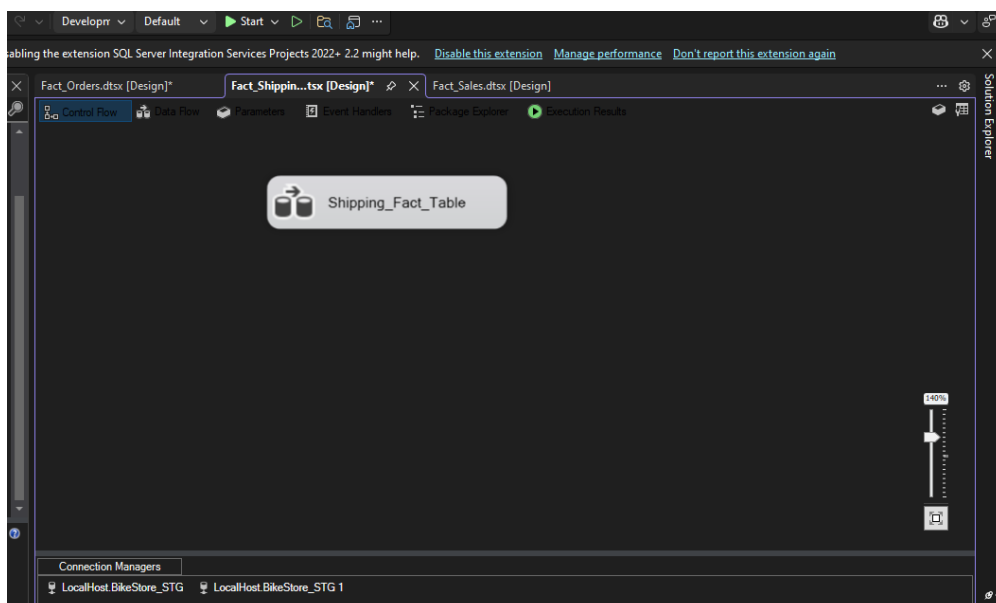
o) **Fact_Sales.dtsx:**

- **Flow Type:** Data flow
- **Description:** Records individual product sales events to track detailed revenue and discount metrics. The SSIS ETL pipeline extracts data from staging, computes net revenue directly at the source engine, and enforces Referential Integrity using sequential Lookup Transformations against the customer, product, geography, and date dimensions. Only verified transactions pass through the Lookup Match Output, where operational IDs are swapped for optimized data warehouse to surrogate keys before being loaded into the destination table for executive KPI reporting.



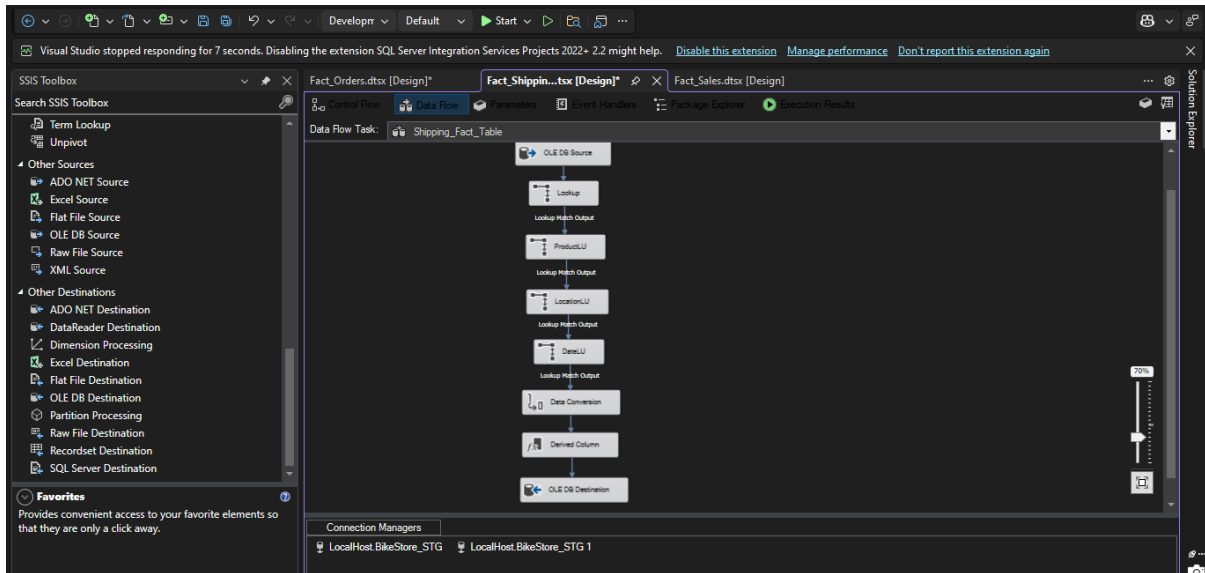
p) **Fact_Shipping.dtsx:**

- **Flow Type:** Control flow



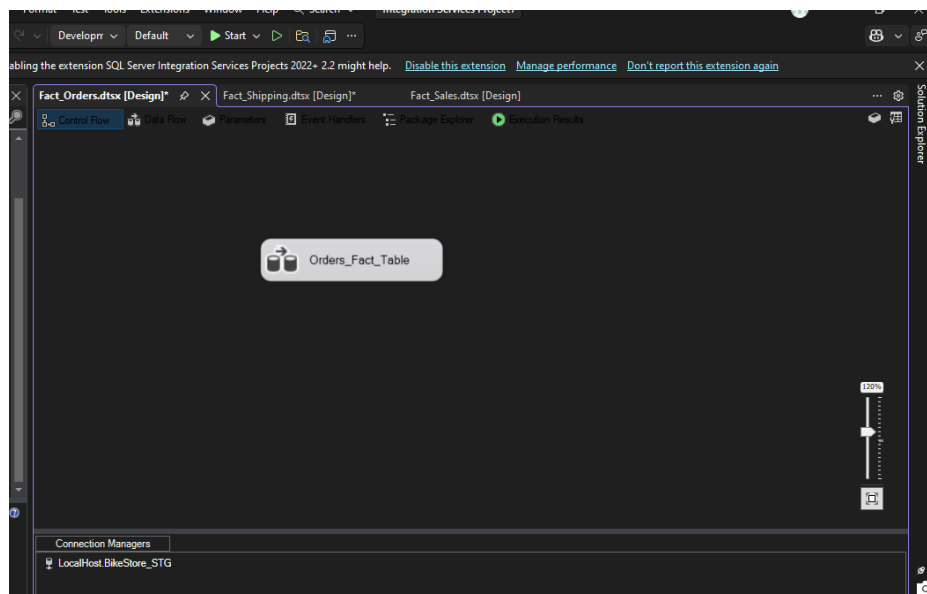
q) Fact_Shipping.dtsx:

- **Flow Type:** Data flow
- **Description:** Captures the logistics and fulfillment lifecycle to measure delivery performance. The SSIS ETL pipeline extracts delivery records from the staging layer and calculates operational time, such as the actual processing days and late shipment flags



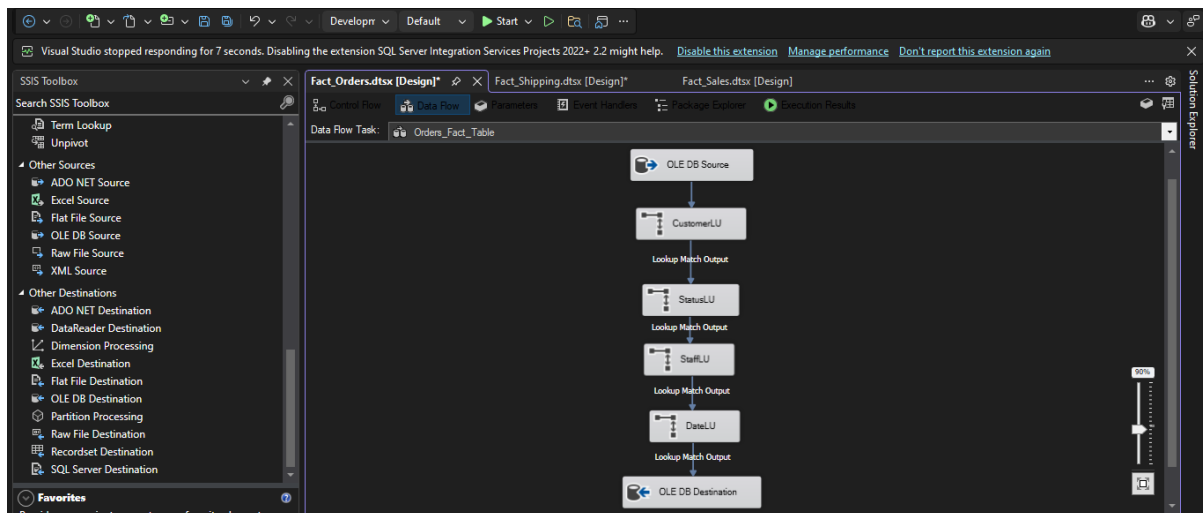
r) Fact_Order.dtsx:

- **Flow Type:** Control flow



s) Fact_Order.dtsx:

- **Flow Type:** Data flow
- **Description:** It summarizes the financial and status outcomes of the entire transactions. The SSIS pipeline uses an aggregation method to on individual line items, calculating total order value and total item counts. Verified orders are mapped to the data warehouse to surrogate keys and loaded into the destination table to track KPIs like average order value, basket size, and order success rates.



4. Queries on each fact table

- **Fact_Sales queries:**
 - **Top Selling Products by Quantity.**

SQLQuery1.sql - (...UHANNAD\User (58)) -> Muhannad\MSSQLSE...bo.Fact_Shipping

```
-- Fact_Sales queries
-- Top Selling Products by Quantity
SELECT p.Product_Name, SUM(fs.Quantity) AS TotalUnitsSold
FROM Fact_Sales fs JOIN Dim_Product p ON fs.ProductKey = p.ProductKey
GROUP BY p.Product_Name ORDER BY TotalUnitsSold DESC;
```

	Product_Name	TotalUnitsSold
1	Electra Cruiser 1 (24-Inch) - 2016	296
2	Electra Townie Original 7D EQ - 2016	290
3	Electra Townie Original 21D - 2016	289
4	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	269
5	Surly Ice Cream Truck Frameset - 2016	167
6	Electra Girl's Hawaii 1 (20-inch) - 2015/2016	154
7	Trek Slash 8 27.5 - 2016	154
8	Surly Straggler 650b - 2016	151
9	Electra Townie Original 7D - 2015/2016	148
10	Surly Straggler - 2016	147
11	Trek Conduit+ - 2016	145
12	Trek Fuel EX 8 29 - 2016	143
13	Electra Moto 1 - 2016	137
14	Pure Cycles Western 3-Speed - Women's - ...	134
15	Pure Cycles Vine 8-Speed - 2016	133
16	Heller Shagamaw Frame - 2016	129
17	Surly Wednesday Frameset - 2016	126
18	Trek Remedy 29 Carbon Frameset - 2016	125
19	Electra Townie Original 7D EQ - Women's ...	125
20	Ritchey Timberwolf Frameset - 2016	118
21	Pure Cycles William 3-Speed - 2016	109

- **Total Net Revenue by Brand.**

SQLQuery1.sql - (...UHANNAD\User (58))* -> X Muhannad\MSSQLSE...bo.Fact_Shipping

```
-- Total Net Revenue by Brand
SELECT p.Brand_Name, SUM(fs.Net_Revenue) AS TotalNetRevenue
FROM Fact_Sales fs JOIN Dim_Product p ON fs.ProductKey = p.ProductKey
GROUP BY p.Brand_Name ORDER BY TotalNetRevenue DESC;
```

100 %

Results Messages

	Brand_Name	TotalNetRevenue
1	Trek	4602752.41
2	Electra	1205318.10
3	Surly	949506.31
4	Sun Bicycles	341994.16
5	Haro	185384.19
6	Heller	171458.93
7	Pure Cycles	149476.34
8	Ritchey	78898.82
9	Strider	4320.45

- **Fact_Shipping queries:-**

- **Late Shipment Rate per Store.**

SQLQuery1.sql - (...UHANNAD\User (58))* -> X Muhannad\MSSQLSE...bo.Fact_Shipping

```
-- Fact_Shipping queries
-- Late Shipment Rate per Store
SELECT g.Store_Name,
COUNT(*) AS TotalShipments,
SUM(fs.LateRiskFlag) AS LateShipments,
CAST(SUM(fs.LateRiskFlag)*100.0/COUNT(*) AS DECIMAL(5,2)) AS LateRate_Pct
FROM Fact_Shipping fs JOIN Dim_Geography g ON fs.LocationKey = g.LocationKey
GROUP BY g.Store_Name;
```

100 %

Results Messages

	Store_Name	TotalShipments	LateShipments	LateRate_Pct
1	Baldwin Bikes	1019	317	31.11
2	Rowlett Bikes	142	37	26.06
3	Santa Cruz Bikes	284	104	36.62

- **Average Fulfillment Time.**

SQLQuery1.sql - (...UHANNAD\User (58))* X Muhannad\MSSQLSE...bo.Fact_Shipping

```
-- Average Fulfillment Time
SELECT AVG(ActualDays) AS AvgFulfillmentDays FROM Fact_Shipping;
```

100 %

Results Messages

	AvgFulfillmentDays
1	1

- **Fact_Orders queries:-**

- **Order Success Rate.**

SQLQuery1.sql - (...UHANNAD\User (58))* X Muhannad\MSSQLSE...bo.Fact_Shipping

```
-- Order Success Rate
SELECT s.Status_Description, COUNT(*) AS OrderCount,
       CAST(COUNT(*)*100.0/(SELECT COUNT(*) FROM Fact_Orders) AS DECIMAL(5,2)) AS Percentage
FROM Fact_Orders fo JOIN Dim_Order_Status s ON fo.StatusKey = s.StatusKey
GROUP BY s.Status_Description;
```

100 %

Results Messages

	Status_Description	OrderCount	Percentage
1	Completed	1445	89.47
2	Pending	62	3.84
3	Processing	63	3.90
4	Rejected	45	2.79

- **Average items count.**

SQLQuery1.sql - (...UHANNAD\User (58))* X Muhannad\MSSQLSE...bo.Fact_Shipping

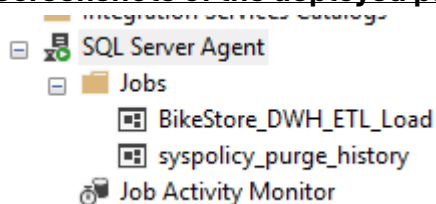
```
-- Average Basket Size
SELECT AVG(CAST(Item_Count AS FLOAT)) AS AvgBasketSize FROM Fact_Orders;
```

100 %

Results Messages

	AvgBasketSize
1	4.38266253869969

5. Screenshots of the deployed packages in SSIS with their schedule.



New Job

Select a page

General

Steps

Schedules

Alerts

Notifications

Targets

Connection

Server:
Muhannad\MSSQLSERVER02

Connection:
MUHANNAD\User

 [View connection properties](#)

Progress

Ready

Script

Help

Name:

BikeStore_DWH_ETL_Load

Owner:

MUHANNAD\User

...

Category:

[Uncategorized (Local)]

▼

...

Description:

☒ Enabled

OK

Cancel

New Job

Select a page

General

Steps

Schedules

Alerts

Notifications

Targets

Script

Help

Job step list:

Step	Name	Type	On Success	On Fail
1	Truncate Tables	Transact-S...	Go to the n...	Quit th
2	Run main package	Transact-S...	Go to the n...	Quit th

Job Step Properties - Truncate Tables

Select a page

General

Advanced

Script

Help

Step name:

Truncate Tables

Type:

Transact-SQL script (T-SQL)

Run as:

Database:

BikeStore_DWH

Command:

DELETE FROM fact_Sales;
DELETE FROM fact_Orders;
DELETE FROM fact_Shipping;
DBCC CHECKIDENT (fact_Sales', RESEED, 0);
DBCC CHECKIDENT (fact_Orders', RESEED, 0);
DBCC CHECKIDENT (fact_Shipping', RESEED, 0);

Open...

Select All

Copy

Paste

Parse

Connection

Server:

Muhannad\MSSQLSERVER02

Connection:

MUHANNAD\User

View connection properties

Progress

Ready

Previous

Next

New Job

Select a page

General

Steps

Schedules

Alerts

Notifications

Targets

Script

Help

Job step list:

Step	Name	Type	On Success	On Fail
1	Truncate Tables	Transact-S...	Go to the n...	Quit th
2	Run main package	Transact-S...	Go to the n...	Quit th

Job Step Properties - Run main package

Select a page

General

Advanced

Script

Help

Step name:

Run main package

Type:

Operating system (CmdExec)

Run as:

SQL Server Agent Service Account

Process exit code of a successful command:

0

Command:

xec /f C:\Users\User\Desktop\Analytics-Warehouse\Analytics-Warehouse

Open...

Select All

Copy

Paste

Tip: Enclose command names that contain spaces in quotation marks. For example:
"command name" <arguments>.

Previous

Next

Connection

Server:

Muhannad\MSSQLSERVER02

Connection:

MUHANNAD\User

View connection properties

Progress

Ready

New Job

New Job Schedule

Name: Daily Load Jobs in Schedule

Schedule type: Recuring ☒ Enabled

One-time occurrence

Date: 2026/05/10 Time: 10:23:29

Frequency

Occurs: Daily

Recurs every: 1 day(s)

Daily frequency

☒ Occurs once at: 12:00:00

☐ Occurs every: 1 hour(s) Starting at: 12:00:00 Ending at: 11:59:59

Duration

Start date: 2026/05/10 ☐ End date: 2026/05/10 ☒ No end date:

Summary

Description: Occurs every day at 12:00:00. Schedule will be used starting on 10/05/2026.

OK Cancel Help

Start Jobs - MUHANNAD

 **Success** 2 Total
2 Success

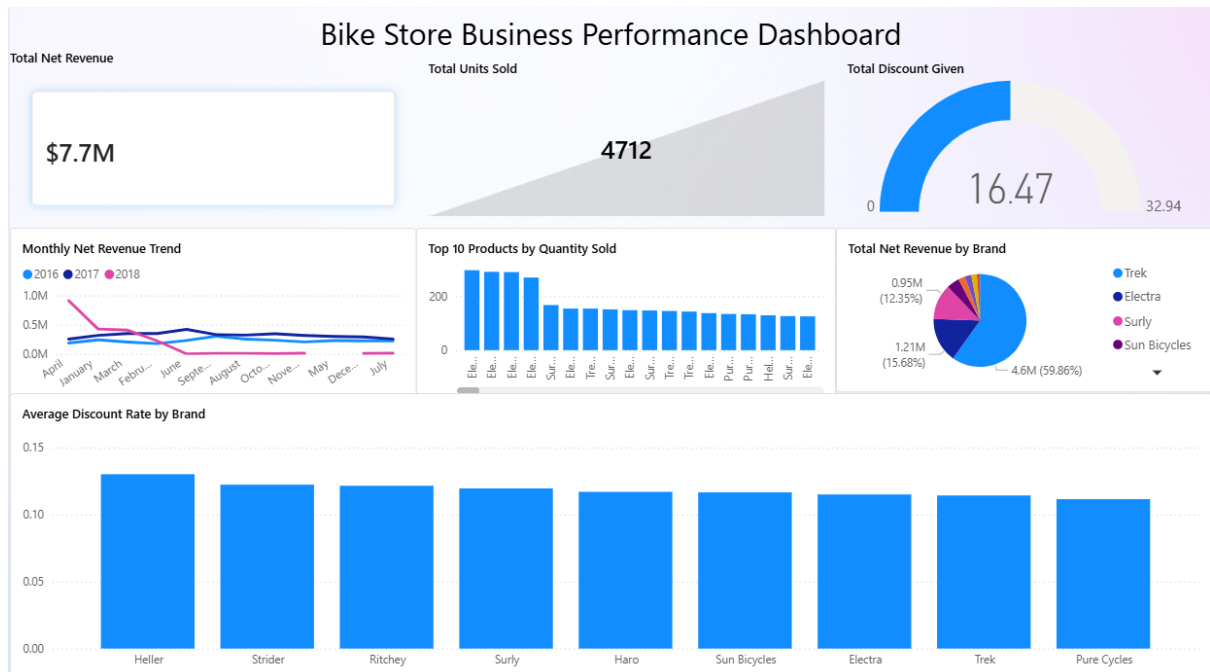
Details:

Action	Status	Message
 Start Job 'BikeStore_DWH_ETL_Load'	Success	
 Execute job 'BikeStore_DWH_ETL_Load'	Success	

Close

6. Build an interactive dashboard for the DWH

Sales Performance Page:



Shipping & Logistics Page:



Order Management Page:

